머클 트리 검증 시스템 분석

개념적 배경

왜 머클 트리가 필요한가?

머클 트리(Merkle Tree)는 Ethereum 블록체인에서 데이터 무결성과 효율적인 검증을 위해 사용되는 핵심 데이터 구조입니다. 다음과 같은 이유로 필요합니다:

1. **데이터 무결성**: 대량의 데이터를 단일 해시로 요약하여 변조 감지
2. **효율적 검증**: 전체 데이터 없이도 특정 데이터의 포함 여부 확인 가능
3. **저장 공간 절약**: 블록 헤더에 루트 해시만 저장하여 공간 효율성 확보
4. **빠른 동기화**: 라이트 클라이언트가 전체 블록 다운로드 없이 검증 가능

수학적 원리

머클 트리는 이진 트리 구조로, 각 리프 노드는 데이터의 해시이고, 내부 노드는 자식 노드들의 해시를 연결한 해시입니다:

```
Root Hash

/ \

H(A,B) H(C,D)

/ \ / \

H(A) H(B) H(C) H(D)

| | | |
```

```
A B C D
```

여기서 H는 Keccak256 해시 함수입니다.

구현 분석

핵심 데이터 구조

1. 블록 헤더의 머클 루트들

```go
type Header struct {
Root common.Hash `json:"stateRoot" gencodec:"required"`
TxHash common.Hash `json:"transactionsRoot" gencodec:"required"`
ReceiptHash common.Hash `json:"receiptsRoot" gencodec:"required"`
UncleHash common.Hash `json:"sha3Uncles" gencodec:"required"`
// ... 기타 필드들
}
```

2. DerivableList 인터페이스

```go
type DerivableList interface {
Len() int
EncodeIndex(int, *bytes.Buffer)
}
```

3. TrieHasher 인터페이스

```go
type TrieHasher interface {
Reset()
Update([]byte, []byte) error
Hash() common.Hash
}
```

주요 알고리즘

1. 머클 루트 계산 (DeriveSha)

```go
func DeriveSha(list DerivableList, hasher TrieHasher) common.Hash {
hasher.Reset()
valueBuf := encodeBufferPool.Get().(*bytes.Buffer)
defer encodeBufferPool.Put(valueBuf)
// StackTrie는 해시 순서대로 삽입되어야 함
// 특별한 삽입 순서로 올바른 순서 보장
var indexBuf []byte
// 인덱스 1부터 0x7f까지 먼저 삽입
for i := 1; i < list.Len() && i <= 0x7f; i++ {
indexBuf = rlp.AppendUint64(indexBuf[:0], uint64(i))
value := encodeForDerive(list, i, valueBuf)
hasher.Update(indexBuf, value)
}
// 인덱스 0 삽입
```

```
if list.Len() > 0 {
indexBuf = rlp.AppendUint64(indexBuf[:0], 0)
value := encodeForDerive(list, 0, valueBuf)
hasher.Update(indexBuf, value)
}
// 인덱스 0x80부터 나머지 삽입
for i := 0x80; i < list.Len(); i++ {
indexBuf = rlp.AppendUint64(indexBuf[:0], uint64(i))
value := encodeForDerive(list, i, valueBuf)
hasher.Update(indexBuf, value)
}
return hasher.Hash()
}
```

2. StackTrie 구현

```
type StackTrie struct {
root *stNode
h *hasher
last []byte
onTrieNode OnTrieNode
kBuf []byte
pBuf []byte
```

```go
}


func (t *StackTrie) Update(key, value []byte) error {
if len(value) == 0 {
return errors.New("trying to insert empty (deletion)")
}
t.grow(key)
k := writeHexKey(t.kBuf, key)
// 키는 오름차순으로 삽입되어야 함
if bytes.Compare(t.last, k) >= 0 {
return errors.New("non-ascending key order")
}
if t.last == nil {
t.last = append([]byte{}, k...)
} else {
t.last = append(t.last[:0], k...)
}
t.insert(t.root, k, value, t.pBuf[:0])
return nil
}
```

3. 트랜잭션 머클 트리 구현

```go
func (s Transactions) EncodeIndex(i int, w *bytes.Buffer) {
```

```
tx := s[i]
if tx.Type() == LegacyTxType {
rlp.Encode(w, tx.inner)
} else {
tx.encodeTyped(w)
}
}
```

4. 영수증 머클 트리 구현

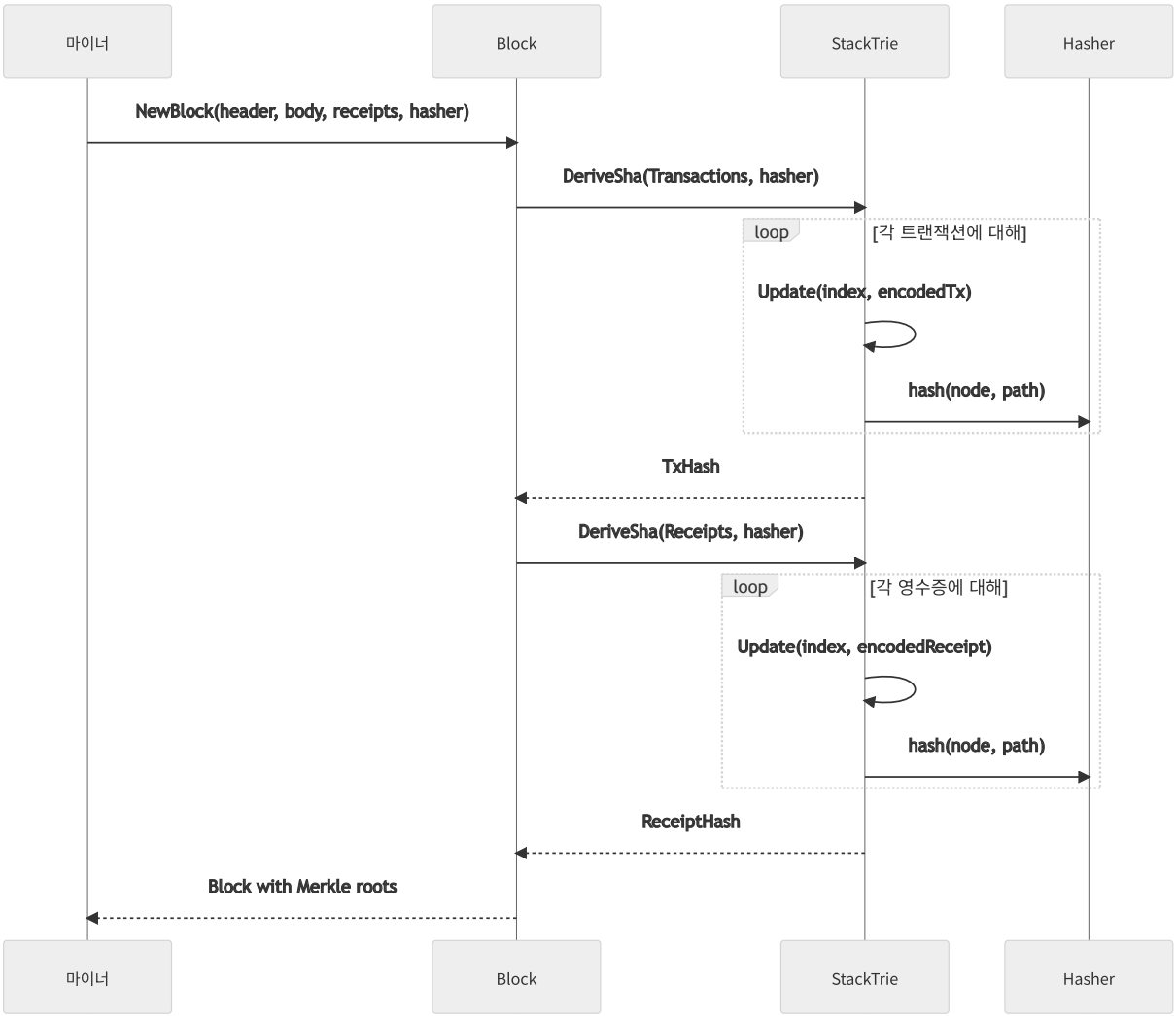
```
func (rs Receipts) EncodeIndex(i int, w *bytes.Buffer) {
r := rs[i]
data := &receiptRLP{
r.statusEncoding(),
r.CumulativeGasUsed,
r.Bloom,
r.Logs,
}
rlp.Encode(w, data)
}
```

코드 구조 및 모듈 관계

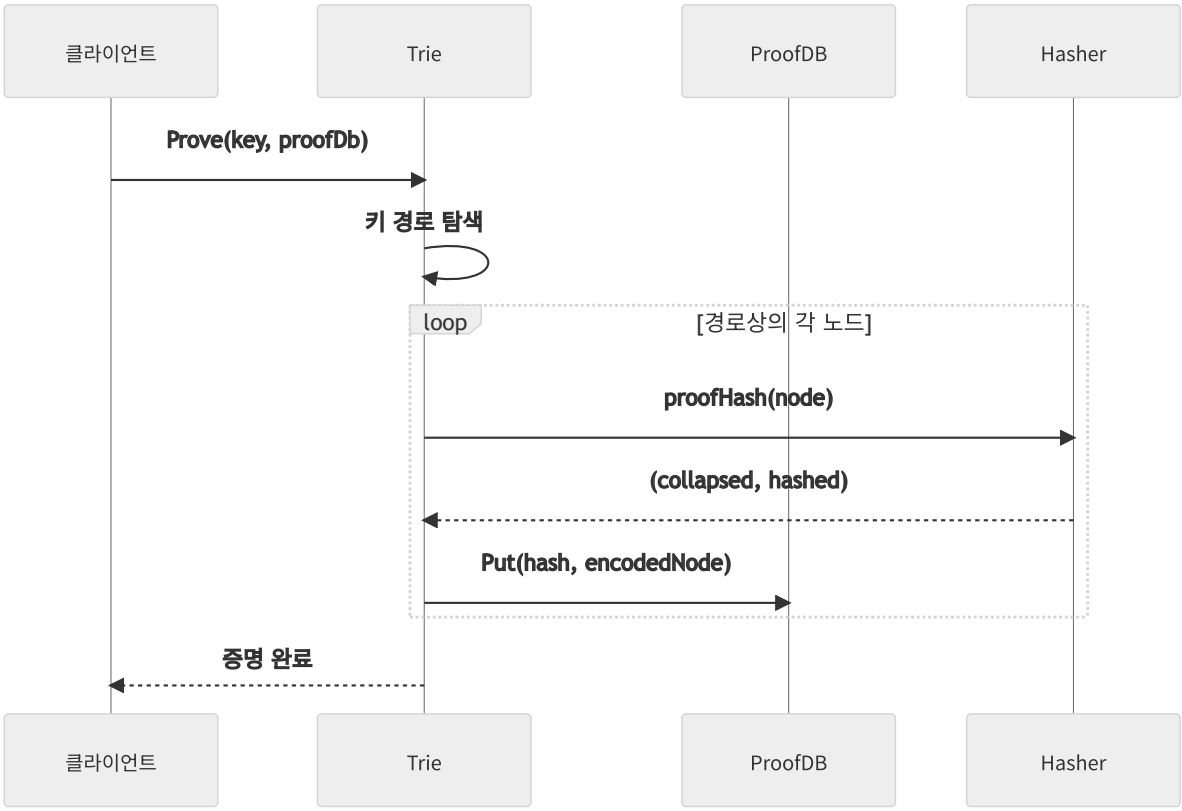
```
core/types/
├── hashing.go # 머클 루트 계산 로직
├── block.go # 블록 헤더 머클 루트 필드
├── transaction.go # 트랜잭션 DerivableList 구현
├── receipt.go # 영수증 DerivableList 구현
└── withdrawal.go # 출금 DerivableList 구현

trie/
├── stacktrie.go # 효율적인 머클 트리 구축
├── hasher.go # 트라이 노드 해싱
├── proof.go # 머클 증명 생성/검증
└── trie.go # 일반적인 트라이 구현
```

실행 흐름

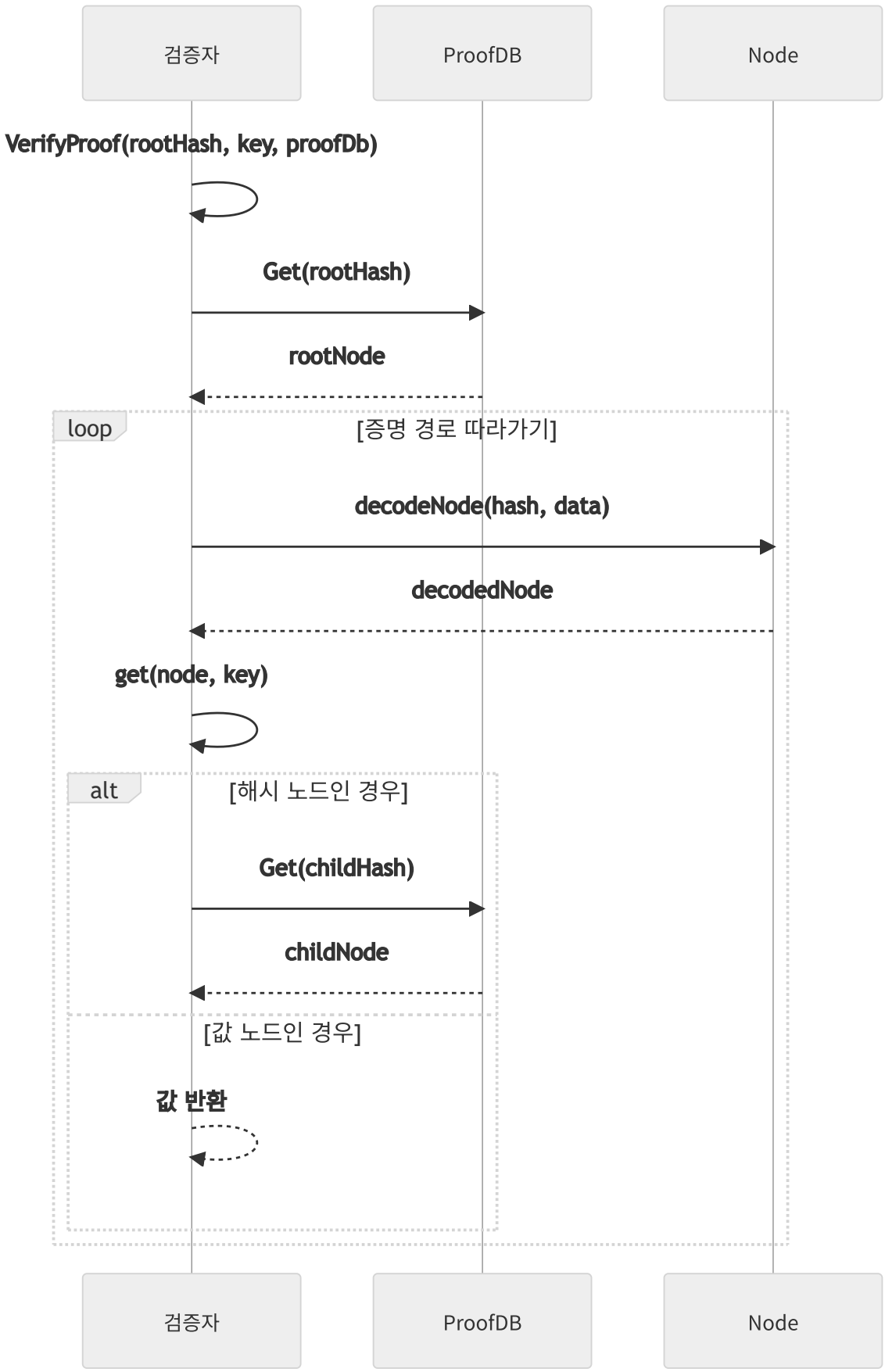
1. 블록 생성 시 머클 루트 계산


마이너
Block
StackTrie
Hasher
NewBlock(header, body, receipts, hasher)
DeriveSha(Transactions, hasher)
loop
[각 트랜잭션에 대해]
Update(index, encodedTx)
hash(node, path)
TxHash
DeriveSha(Receipts, hasher)
loop
[각 영수증에 대해]
Update(index, encodedReceipt)
hash(node, path)
ReceiptHash
Block with Merkle roots
마이너
Block
StackTrie
Hasher

2. 머클 증명 생성 과정


클라이언트
Trie
ProofDB
Hasher
Prove(key, proofDb)
키 경로 탐색
loop
[경로상의 각 노드]
proofHash(node)
(collapsed, hashed)
Put(hash, encodedNode)
증명 완료
클라이언트
Trie
ProofDB
Hasher

3. 머클 증명 검증 과정


검증자
ProofDB
Node
VerifyProof(rootHash, key, proofDb)
Get(rootHash)
rootNode
loop
[증명 경로 따라가기]
decodeNode(hash, data)
decodedNode
get(node, key)
alt
[해시 노드인 경우]
Get(childHash)
childNode
[값 노드인 경우]
값 반환
검증자
ProofDB
Node

4. 단계별 실행 과정

트랜잭션 머클 루트 계산:

1. **트랜잭션 인코딩**: 각 트랜잭션을 RLP 인코딩
2. **인덱스 생성**: 트랜잭션 인덱스를 RLP 인코딩하여 키로 사용
3. **StackTrie 삽입**: 특별한 순서로 (key, value) 쌍 삽입
4. **노드 해싱**: 각 내부 노드를 Keccak256으로 해싱
5. **루트 계산**: 최종 루트 해시 반환

영수증 머클 루트 계산:

1. **영수증 인코딩**: 각 영수증을 receiptRLP 구조로 인코딩
2. **인덱스 매핑**: 영수증 인덱스를 키로 사용
3. **트리 구축**: StackTrie를 사용하여 효율적으로 구축
4. **해시 계산**: 내부 노드들의 해시 계산
5. **루트 도출**: 최종 영수증 루트 해시

성능 최적화

1. StackTrie 최적화

```
type StackTrie struct {
root *stNode
h *hasher
last []byte  // 마지막 삽입된 키 캐싱
onTrieNode OnTrieNode // 노드 커밋 콜백
kBuf []byte  // 키 버퍼 재사용
```

```
pBuf []byte  // 경로 버퍼 재사용
}

func (t *StackTrie) grow(key []byte) {
// 버퍼 크기 동적 조정
if  cap(t.kBuf) < 2*len(key) {
t.kBuf = make([]byte, 2*len(key))
}
if  cap(t.pBuf) < 2*len(key) {
t.pBuf = make([]byte, 2*len(key))
}
}
```

2. 메모리 풀링

```
var (
stPool = sync.Pool{New: func() any { return  new(stNode) }}
bPool = newBytesPool(32, 100)
)

// 노드 재사용
func (n *stNode) reset() *stNode {
if  n.typ == hashedNode {
```

```
bPool.Put(n.val) // 해시 노드 값을 풀에 반환

}

n.key = n.key[:0]

n.val = nil

for  i := range  n.children {

n.children[i] = nil

}

n.typ = emptyNode

return  n

}
```

3. 해시 풀링

```
var  hasherPool = sync.Pool{

New: func() interface{} { return  crypto.NewKeccakState() },

}


func  rlpHash(x  interface{}) (h common.Hash) {

sha := hasherPool.Get().(crypto.KeccakState)

defer  hasherPool.Put(sha)

sha.Reset()

rlp.Encode(sha, x)

sha.Read(h[:])
```

```
return h
}
```

4. 버퍼 풀링

```
var encodeBufferPool = sync.Pool{
New: func() interface{} { return new(bytes.Buffer) },
}

func encodeForDerive(list DerivableList, i int, buf *bytes.Buffer) []byte {
buf.Reset()
list.EncodeIndex(i, buf)
// StackTrie가 값을 보유하므로 복사 필요
return common.CopyBytes(buf.Bytes())
}
```

5. 병렬 해싱

```
func (h *hasher) hashFullNodeChildren(n *fullNode) *fullNode {
var children [17]node
if h.parallel {
var wg sync.WaitGroup
for i := 0; i < 16; i++ {
```

```
if child := n.Children[i]; child != nil {
wg.Add(1)
go func(i int) {
hasher := newHasher(false)
children[i] = hasher.hash(child, false)
returnHasherToPool(hasher)
wg.Done()
}(i)
} else {
children[i] = nilValueNode
}
}
wg.Wait()
} else {
// 순차 처리
for i := 0; i < 16; i++ {
if child := n.Children[i]; child != nil {
children[i] = h.hash(child, false)
} else {
children[i] = nilValueNode
}
}
}
```

```
return &fullNode{flags: nodeFlag{}, Children: children}

}
```

실제 사용 예시

1. 블록 생성 시 머클 루트 계산

```
func NewBlock(header *Header, body *Body, receipts []*Receipt, hasher TrieHashe

b := NewBlockWithHeader(header)

// 트랜잭션 머클 루트 계산

if len(body.Transactions) == 0 {

b.header.TxHash = EmptyTxsHash

} else {

b.header.TxHash = DeriveSha(Transactions(body.Transactions), hasher)

b.transactions = make(Transactions, len(body.Transactions))

copy(b.transactions, body.Transactions)

}

// 영수증 머클 루트 계산

if len(receipts) == 0 {

b.header.ReceiptHash = EmptyReceiptsHash

} else {

b.header.ReceiptHash = DeriveSha(Receipts(receipts), hasher)

b.header.Bloom = MergeBloom(receipts)
```

```
}

return b

}
```

2. 머클 증명 생성

```
func GenerateProof(trie *Trie, key []byte) (map[string][]byte, error) {

proof := make(map[string][]byte)

proofDb := &memoryDB{db: proof}

err := trie.Prove(key, proofDb)

if err != nil {

return nil, err

}

return proof, nil

}
```

3. 머클 증명 검증

```
func VerifyTransactionProof(rootHash common.Hash, txIndex int, tx *Transaction

// 트랜잭션 인덱스를 키로 변환

key := rlp.AppendUint64(nil, uint64(txIndex))

// 트랜잭션 인코딩

var buf bytes.Buffer
```

```
if tx.Type() == LegacyTxType {
rlp.Encode(&buf, tx.inner)
} else {
tx.encodeTyped(&buf)
}
expectedValue := buf.Bytes()
// 증명 검증
proofDb := &mapDB{db: proof}
value, err := VerifyProof(rootHash, key, proofDb)
if err != nil {
return false, err
}
return bytes.Equal(value, expectedValue), nil
}
```

4. 라이트 클라이언트 검증

```
func (lc *LightClient) VerifyReceipt(blockHash common.Hash, receiptIndex int, r
// 블록 헤더 가져오기
header, err := lc.GetHeader(blockHash)
if err != nil {
return err
}
```

```go
// 영수증 증명 요청
proof, err := lc.GetReceiptProof(blockHash, receiptIndex)
if err != nil {
return err
}
// 영수증 인코딩
key := rlp.AppendUint64(nil, uint64(receiptIndex))
var buf bytes.Buffer
data := &receiptRLP{
receipt.statusEncoding(),
receipt.CumulativeGasUsed,
receipt.Bloom,
receipt.Logs,
}
rlp.Encode(&buf, data)
expectedValue := buf.Bytes()
// 증명 검증
value, err := VerifyProof(header.ReceiptHash, key, proof)
if err != nil {
return fmt.Errorf("proof verification failed: %v", err)
}
if !bytes.Equal(value, expectedValue) {
return errors.New("receipt data mismatch")
```

```
}
return nil
}
```

에러 처리 메커니즘

1. 증명 생성 에러

```
func (t *Trie) Prove(key []byte, proofDb ethdb.KeyValueWriter) error {
if t.committed {
return ErrCommitted
}
// 키 경로 탐색 중 에러 처리
for len(key) > 0 && tn != nil {
switch n := tn.(type) {
case hashNode:
blob, err := t.reader.node(prefix, common.BytesToHash(n))
if err != nil {
log.Error("Unhandled trie error in Trie.Prove", "err", err)
return err
}
tn = mustDecodeNodeUnsafe(n, blob)
}
```

```
}
return nil
}
```

2. 증명 검증 에러

```
func VerifyProof(rootHash common.Hash, key []byte, proofDb ethdb.KeyValueReader
key = keybytesToHex(key)
wantHash := rootHash
for i := 0; ; i++ {
buf, _ := proofDb.Get(wantHash[:])
if buf == nil {
return nil, fmt.Errorf("proof node %d (hash %064x) missing", i, wantHash)
}
n, err := decodeNode(wantHash[:], buf)
if err != nil {
return nil, fmt.Errorf("bad proof node %d: %v", i, err)
}
// 증명 경로 검증 계속...
}
}
```

3. 범위 증명 검증 에러

```
func VerifyRangeProof(rootHash common.Hash, firstKey []byte, keys [][]byte, val
if len(keys) != len(values) {
return false, fmt.Errorf("inconsistent proof data, keys: %d, values: %d", len(ke
}
// 키 순서 검증
for i := 0; i < len(keys); i++ {
if i < len(keys)-1 {
if bytes.Compare(keys[i], keys[i+1]) >= 0 {
return false, errors.New("range is not monotonically increasing")
}
if bytes.HasPrefix(keys[i+1], keys[i]) {
return false, errors.New("range contains path prefixes")
}
}
if len(values[i]) == 0 {
return false, errors.New("range contains deletion")
}
}
// 증명 검증 계속...
}
```

관련 시스템

의존성 관계

1. **core/types**: 블록, 트랜잭션, 영수증 타입 정의
2. **trie**: 머클 트리 구현 및 증명 시스템
3. **crypto**: Keccak256 해시 함수
4. **rlp**: 데이터 직렬화
5. **ethdb**: 증명 데이터 저장소

상호작용 패턴

- **블록 생성**: 마이너가 새 블록 생성 시 머클 루트 계산
- **블록 검증**: 수신된 블록의 머클 루트 검증
- **라이트 클라이언트**: 머클 증명을 통한 데이터 검증
- **상태 동기화**: 머클 증명을 통한 부분 상태 검증

영향 범위

머클 트리 검증 시스템의 변경은 다음 영역에 영향을 미칩니다:

1. **블록 구조**: 헤더의 머클 루트 필드들
2. **네트워크 프로토콜**: 증명 데이터 전송
3. **저장소**: 트라이 노드 저장 방식
4. **성능**: 블록 생성 및 검증 속도
5. **라이트 클라이언트**: 증명 기반 검증 로직

이 머클 트리 검증 시스템은 Ethereum의 확장성과 보안을 동시에 보장하는 핵심 구성요소로, 전체 블록체인 데이터의 무결성을 효율적으로 검증할 수 있게 합니다.